

ФЕДЕРАЛЬНОЕ АГЕНТСТВО ПО ОБРАЗОВАНИЮ

ГОСУДАРСТВЕННОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ «САМАРСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ имени академика С.П. КОРОЛЁВА»

КАФЕДРА «ТЕХНИЧЕСКАЯ КИБЕРНЕТИКА»

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ

«Объектно-ориентированное программирование»

ЛАБОРАТОРНАЯ РАБОТА №7

Студент Трофимов А. В.

Группа 6301-030301D

Руководитель Борисов Д. С.

Оценка _____

Задание №1

В интерфейсе TabulatedFunction был добавлен необходимый родительский ТИП.

```
package functions;

public interface TabulatedFunction extends Function, Cloneable, Iterable<FunctionPoint> {
    // Задание 4: Добавляем метод clone() в интерфейс
    Object clone();

    int getPointsCount();
    FunctionPoint getPoint(int index);
    void setPoint(int index, FunctionPoint point)
        throws InappropriateFunctionPointException, FunctionPointIndexOutOfBoundsException;
    double getPointX(int index) throws FunctionPointIndexOutOfBoundsException;
    void setPointX(int index, double x)
        throws InappropriateFunctionPointException, FunctionPointIndexOutOfBoundsException;
    double getPointY(int index) throws FunctionPointIndexOutOfBoundsException;
    void setPointY(int index, double y) throws FunctionPointIndexOutOfBoundsException;

    void deletePoint(int index)
        throws FunctionPointIndexOutOfBoundsException, IllegalStateException;
    void addPoint(FunctionPoint point) throws InappropriateFunctionPointException;
}
```

В классе ArrayTabulatedFunction был добавлен метод возвращающий объект итератора, класс итератора был сделан анонимным.

```
@Override
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {
        private int currentIndex = 0;

        @Override
        public boolean hasNext() {
            return currentIndex < pointsCount;
        }

        @Override
        public FunctionPoint next() {
            if (!hasNext()) {
                throw new NoSuchElementException(s: "Нет следующего элемента");
            }
            // Возвращаем копию точки для защиты инкапсуляции
            return new FunctionPoint(points[currentIndex++]);
        }

        @Override
        public void remove() {
            throw new UnsupportedOperationException(message: "Удаление не поддерживается");
        }
    };
}
```

Метод удаления выбрасывает исключение UnsupportedOperationException.

Метод получения следующего элемента выбрасывает исключение NoSuchElementException, если следующего элемента нет.

В LinkedListTabulatedFunction был реализован аналогичный итератор.

```
@Override
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {
        private Node current = head.next;
        private int visited = 0;

        @Override
        public boolean hasNext() {
            return visited < size;
        }

        @Override
        public FunctionPoint next() {
            if (!hasNext()) {
                throw new NoSuchElementException(s: "Нет следующего элемента");
            }
            FunctionPoint point = new FunctionPoint(current.point);
            current = current.next;
            visited++;
            return point;
        }

        @Override
        public void remove() {
            throw new UnsupportedOperationException(message: "Удаление не поддерживается");
        }
    };
}
```

В методе main программы проверил работу итераторов классов табулированных функций.

```
public static void main(String[] args) throws FunctionPointIndexOutOfBoundsException, InappropriateFunctionPointException {
    ArrayTabulatedFunction arrayFunc1 = new ArrayTabulatedFunction(leftX: 0, rightX: 9, pointsCount: 11);
    LinkedListTabulatedFunction listFunc1 = new LinkedListTabulatedFunction(leftX: 4, rightX: 10, pointsCount: 9);

    System.out.println(x: "\nArrayTabulatedFunction\n");
    for (FunctionPoint p : arrayFunc1) {
        System.out.println(p);
    }
    System.out.println(x: "\nLinkedListTabulatedFunction\n");
    for (FunctionPoint p : listFunc1) {
        System.out.println(p);
    }
    System.out.println(x: "\n-----\n");
}
```

```

ArrayTabulatedFunction

(0.0; 0.0)
(0.9; 0.0)
(1.8; 0.0)
(2.7; 0.0)
(3.6; 0.0)
(4.5; 0.0)
(5.4; 0.0)
(6.3; 0.0)
(7.2; 0.0)
(8.1; 0.0)
(9.0; 0.0)

LinkedListTabulatedFunction

(4.0; 0.0)
(4.75; 0.0)
(5.5; 0.0)
(6.25; 0.0)
(7.0; 0.0)
(7.75; 0.0)
(8.5; 0.0)
(9.25; 0.0)
(10.0; 0.0)

```

Задание №2

В пакете functions был описан базовый интерфейс фабрик табулированных функций TabulatedFunctionFactory. В интерфейс были добавлены три перегруженных метода.

```

package functions;

// Задание 2: Интерфейс фабрики табулированных функций
public interface TabulatedFunctionFactory {
    TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount);
    TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values);
    TabulatedFunction createTabulatedFunction(FunctionPoint[] points);
}

```

В ArrayTabulatedFunction и LinkedListTabulatedFunction были описаны классы фабрик.

```

public static class ArrayTabulatedFunctionFactory implements TabulatedFunctionFactory {
    @Override
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new ArrayTabulatedFunction(leftX, rightX, pointsCount);
    }

    @Override
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new ArrayTabulatedFunction(leftX, rightX, values);
    }

    @Override
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new ArrayTabulatedFunction(points);
    }
}

```

```

public static class LinkedListTabulatedFunctionFactory implements TabulatedFunctionFactory {
    @Override
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new LinkedListTabulatedFunction(leftX, rightX, pointsCount);
    }

    @Override
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new LinkedListTabulatedFunction(leftX, rightX, values);
    }

    @Override
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new LinkedListTabulatedFunction(points);
    }
}

```

В классе TabulatedFunctions было объявлено приватное статическое поле типа TabulatedFunctionFactory и проинициализированно его объектом ArrayTabulatedFunction фабрик.

```

public class TabulatedFunctions {
    // Приватный конструктор, чтобы нельзя было создать экземпляра класса
    private TabulatedFunctions() {}

    // Задание 2: Фабрика по умолчанию (используем ArrayTabulatedFunction)
    private static TabulatedFunctionFactory factory = new ArrayTabulatedFunction.ArrayTabulatedFunctionFactory();
}

```

Также был объявлен метод setTabulatedFunctionFactory, позволяющий заменить объект фабрики.

```

public static void setTabulatedFunctionFactory(TabulatedFunctionFactory factory) {
    TabulatedFunctions.factory = factory;
}

```

В классе TabulatedFunctions были описаны три перегруженных метода TabulatedFunction createTabulatedFunction, возвращающих объекты табулированных функций, созданные с помощью текущей фабрики.

```

public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
    return factory.createTabulatedFunction(leftX, rightX, pointsCount);
}

public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
    return factory.createTabulatedFunction(leftX, rightX, values);
}

public static TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
    return factory.createTabulatedFunction(points);
}

```

В остальных методах (tabulate, inputTabulatedFunction, readTabulatedFunction) класса было заменено явное создание объектов с помощью конструкторов на вызов метода createTabulatedFunction.

```
return createTabulatedFunction(points);
```

В методе main проверил работу фабрик.

```
class functions.ArrayTabulatedFunction  
class functions.LinkedListTabulatedFunction  
class functions.ArrayTabulatedFunction
```

Задание №3

В классе TabulatedFunctions были добавлены ещё три перегруженных версии метода createTabulatedFunction. Эти методы получают ссылку типа Class на описание класса, объект которого требуется создать. Было сделано так, чтобы в эти методы можно было передать только ссылки на классы, реализующие интерфейс TabulatedFunction.

```
public static TabulatedFunction createTabulatedFunction(Class<? extends TabulatedFunction> functionClass,  
                                                       double leftX, double rightX, int pointsCount) {  
    try {  
        // Получаем конструктор с параметрами (double, double, int)  
        Constructor<? extends TabulatedFunction> constructor =  
            functionClass.getConstructor(...parameterTypes: double.class, double.class, int.class);  
        // Создаем объект  
        return constructor.newInstance(leftX, rightX, pointsCount);  
    } catch (NoSuchMethodException | SecurityException | InstantiationException |  
            IllegalAccessException | IllegalArgumentException | InvocationTargetException e) {  
        throw new IllegalArgumentException(message: "Ошибка при создании объекта через рефлексию", e);  
    }  
}  
  
public static TabulatedFunction createTabulatedFunction(Class<? extends TabulatedFunction> functionClass,  
                                                       double leftX, double rightX, double[] values) {  
    try {  
        // Получаем конструктор с параметрами (double, double, double[])  
        Constructor<? extends TabulatedFunction> constructor =  
            functionClass.getConstructor(...parameterTypes: double.class, double.class, double[].class);  
        // Создаем объект  
        return constructor.newInstance(leftX, rightX, values);  
    } catch (NoSuchMethodException | SecurityException | InstantiationException |  
            IllegalAccessException | IllegalArgumentException | InvocationTargetException e) {  
        throw new IllegalArgumentException(message: "Ошибка при создании объекта через рефлексию", e);  
    }  
}  
  
public static TabulatedFunction createTabulatedFunction(Class<? extends TabulatedFunction> functionClass,  
                                                       FunctionPoint[] points) {  
    try {  
        // Получаем конструктор с параметрами (FunctionPoint[])  
        Constructor<? extends TabulatedFunction> constructor =  
            functionClass.getConstructor(...parameterTypes: FunctionPoint[].class);  
        // Создаем объект  
        return constructor.newInstance((Object)points);  
    } catch (NoSuchMethodException | SecurityException | InstantiationException |  
            IllegalAccessException | IllegalArgumentException | InvocationTargetException e) {  
        throw new IllegalArgumentException(message: "Ошибка при создании объекта через рефлексию", e);  
    }  
}
```

Методы находят в классе конструктор с соответствующими определенными параметрами (левая граница, правая граница, количество точек, массив точек Y или массив точек), корректно определяет тип и создает объект табулированной функции, после возвращает ссылку на этот объект.

В классе `TabulatedFunctions` были перегружены методы (`tabulate`, `inputTabulatedFunction`, `readTabulatedFunction`) создающие объекты табулированных функций, добавив версии, принимающие также ссылку типа `Class` на описание класса, объект которого требуется создать.

Было сделано так, чтобы в эти методы можно было передать только ссылки на классы, реализующие интерфейс `TabulatedFunction`.

```
public static TabulatedFunction tabulate(Class<? extends TabulatedFunction> functionClass,  
                                         Function function, double leftX, double rightX, int pointsCount) {  
    // Проверка, что интервал табулирования находится в области определения функции  
    if (leftX < function.getLeftDomainBorder() || rightX > function.getRightDomainBorder()) {  
        throw new IllegalArgumentException(s: "Интервал табулирования находится за пределами области определения функции");  
    }  
}
```

```
public static TabulatedFunction inputTabulatedFunction(Class<? extends TabulatedFunction> functionClass,  
                                                       InputStream in) throws IOException {  
    DataInputStream dis = new DataInputStream(in);  
}
```

```
public static TabulatedFunction readTabulatedFunction(Class<? extends TabulatedFunction> functionClass,  
                                                      Reader in) throws IOException {  
    StreamTokenizer tokenizer = new StreamTokenizer(in);  
}
```

Методы вызывают соответствующий метод `createTabulatedFunction`, передавая на него ссылку типа `Class` и массив `points`.

```
return createTabulatedFunction(functionClass, points);
```

Проверил в методе `main` работу методов рефлексивного создания объектов, а также методов класса `TabulatedFunctions`, использующих создание объектов.

```
class functions.ArrayTabulatedFunction  
{(0.0; 0.0), (5.0; 0.0), (10.0; 0.0)}  
class functions.ArrayTabulatedFunction  
{(0.0; 0.0), (10.0; 10.0)}  
class functions.LinkedListTabulatedFunction  
{(0.0; 0.0), (10.0; 10.0)}  
class functions.LinkedListTabulatedFunction  
{(0.0; 0.0), (0.3141592653589793; 0.3090169943749474), (0.6283185307179586; 0.5877852522924731), (0.9424777960769379; 0.8090169943749475), (1.2566370614359172; 0.5877852522924732), (2.827433388230814; 0.3090169943749475), (3.141592653589793; 1.2246467991473532E-16)}
```